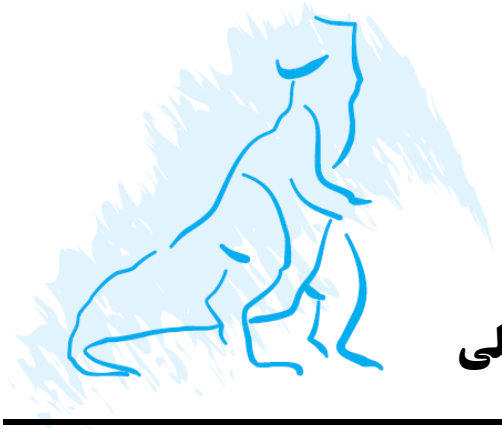


فصل ۹

حافظه‌ی اصلی

در فصل ۵، نشان دادیم که پردازنده چگونه می‌تواند توسط مجموعه‌ای از پردازه‌ها به اشتراک گرفته شود. به عنوان نتیجه‌ای از زمانبندی پردازنده می‌توانیم هر دوی بهره‌وری پردازنده و سرعت پاسخگویی کامپیوتر به کاربران خود را بهبود ببخشیم. با این وجود، جهت تحقق افزایش کارایی بایستی پردازه‌های زیادی را در داخل حافظه نگه بداریم–یعنی، باید حافظه را به اشتراک بگذاریم.

در این فصل، شیوه‌های گوناگون مدیریت حافظه را بحث می‌کنیم. الگوریتم‌های مدیریت حافظه از روش ابتدایی ماشین برهنه تا یک راهبرد استفاده‌کننده از صفحه‌بندی را شامل می‌شوند. هر رویکرد، مزایا و معایب خاص خود را دارد. انتخاب یک روش مدیریت حافظه برای یک سیستم خاص، به عوامل متعددی مخصوصاً طراحی سخت‌افزار سیستم بستگی دارد. چنان که خواهیم دید، اکثر الگوریتم‌ها نیازمند پشتیبانی سخت‌افزار هستند که باعث می‌شود سیستم‌های زیادی، سخت‌افزار و مدیریت حافظه‌ی سیستم‌عامل را نزدیک هم قرار بدهند.

اهداف فصل

- تشریح تفاوت میان یک آدرس منطقی و فیزیکی و نقش واحد مدیریت حافظه (MMU) در ترجمه‌ی آدرس‌ها.
- به‌کارگیری راهبردهای اول، بهترین و بدترین گنجایش برای تخصیص همجوار حافظه.
- تشریح تمایز میان پارگی داخلی و خارجی.
- ترجمه‌ی آدرس‌های منطقی به فیزیکی در یک سیستم صفحه‌بندی که شامل یک بافر ترجمه در کنار (TLB) می‌باشد.
- تبیین صفحه‌بندی سلسله‌مراتبی، صفحه‌بندی درهم‌شده، و جدول‌های صفحه‌ی وارونه.
- تبیین ترجمه‌ی آدرس برای معماری‌های IA-32، x86-64، و ARMv8.

۹-۱ پس‌زمینه

همان‌گونه که در فصل ۱ دیدیم، حافظه در عملیات سیستم کامپیوتری مدرن جایگاه مرکزی دارد. حافظه شامل آرایه‌ی بزرگی از بایت‌هاست که هر یک آدرس خاص خود را دارند. پردازنده دستورالعمل‌ها را مطابق مقدار شمارنده‌ی برنامه از حافظه واکشی می‌کند. این

دستورالعمل‌ها نیز می‌توانند آدرس‌های خاص دیگری را از حافظه بارگذاری کرده و یا در آنها ذخیره کنند.

دور نوعی اجرای دستورالعمل، نخست دستورالعمل را از حافظه واکشی می‌کند. سپس دستورالعمل رمزگشایی شده و می‌تواند منجر به واکشی عملوندها از حافظه گردد. بعد از این که دستورالعمل بر روی عملوندها اجرا شد، نتایج آن می‌تواند دوباره در حافظه ذخیره شود. واحد حافظه فقط جریانی از آدرس‌های حافظه را می‌بیند؛ نمی‌داند که این آدرس‌ها چگونه تولید شده‌اند (با شمارنده‌ی دستورالعمل، ایندکس، غیر مستقیم، آدرس‌های ثابت و از این دست) یا برای چه منظوری ارائه شده‌اند (دستورالعمل‌ها یا داده‌ها). بر این اساس، می‌توانیم از **چگونگی** تولید آدرس‌های حافظه توسط یک برنامه چشم‌پوشی کنیم. تنها به دنباله‌ی آدرس‌های حافظه‌ای که توسط برنامه‌ی در حال اجرا تولید می‌شوند علاقه داریم.

بحث خود را با طرح ملاحظات متعددی که به مدیریت حافظه مربوط می‌شوند آغاز می‌کنیم: سخت‌افزار پایه، پیوند آدرس‌های نمادین (یا مجازی) حافظه به آدرس‌های فیزیکی واقعی و تمایز آدرس‌های فیزیکی و منطقی. بخش را با بحثی از اتصال پویا و کتابخانه‌های مشترک جمع می‌بندیم.

۹-۱-۱ سخت‌افزار پایه

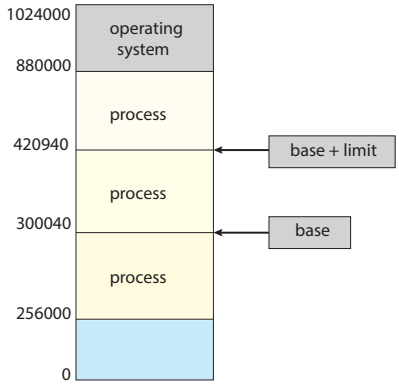
حافظه‌ی اصلی و ثبات‌های ساخته شده در هر هسته‌ی پردازشی تنها حافظه‌ی همه‌منظوره‌ای هستند که پردازنده می‌تواند مستقیماً دستیابی کند. دستورالعمل‌هایی از ماشین هستند که آدرس‌های حافظه را به عنوان آرگومان می‌پذیرند، اما هیچ یک از آنها آدرس‌های دیسک را بر نمی‌دارند. بنابراین، هر دستورالعمل اجرایی و هرگونه داده‌ای که توسط این دستورالعمل‌ها استفاده می‌شوند، بایستی در یکی از این دو دستگاه حافظه‌ی با دسترسی مستقیم قرار بگیرند. اگر داده‌ها در حافظه نباشند، پیش از آنکه پردازنده بتواند روی آنها عملیاتی داشته باشد بایستی در آنجا قرار بگیرند.

ثبات‌هایی که در داخل هر هسته‌ی پردازنده ساخته می‌شوند عموماً با یک دور ساعت پردازنده قابل دسترسی هستند. برخی هسته‌های پردازنده می‌توانند با هر تیک ساعت، دستورالعمل‌های یک یا چند عملیات را رمزگشایی کرده و عملیات ساده‌ای بر روی محتوای ثبات‌ها انجام دهند. این گفته در مورد حافظه‌ی اصلی که از طریق تراکنشی روی گذرگاه حافظه دسترسی می‌شود، صادق نیست. تکمیل دسترسی حافظه می‌تواند دورهای زیادی را از ساعت پردازنده بگیرد. در اینجا، پردازنده اصولاً می‌ایستد، چون داده‌های مورد نیاز خود را برای تکمیل دستورالعمل در حال اجرا ندارد. این وضعیت به دلیل دفعات مکرر دسترسی‌های حافظه قابل تحمل نیست. چاره آن است که حافظه‌ی پرسرعتی مابین پردازنده و حافظه‌ی اصلی، نوعاً بر روی تراشه‌ی پردازنده جهت دسترسی سریع افزوده شود. **کَش** این چنینی در بخش ۱-۵-۵ مطرح شد. سخت‌افزار به منظور مدیریت کَش که در داخل پردازنده ساخته شده، دسترسی حافظه را به صورت خودکار بی‌آنکه کنترلی از جانب سیستم‌عامل باشد، تسریع می‌بخشد. (از بخش ۵-۵-۲

به‌خاطر بیاورید که در طی یک ایست حافظه، یک هسته‌ی چندنخی می‌تواند از نخ سخت‌افزاری ایستاده به نخ سخت‌افزاری دیگری سوییچ نماید.)

نه تنها نگران سرعت نسبی دسترسی حافظه‌ی فیزیکی هستیم، بلکه بایستی صحت عملیات را تضمین کنیم. برای عملیات درست سیستم می‌بایست سیستم‌عامل را از دسترسی پردازه‌های کاربر، همین‌طور پردازه‌های کاربر را از یکدیگر محافظت کنیم. این حفاظت باید توسط سخت‌افزار فراهم شود، چون سیستم‌عامل معمولاً میان پردازنده و دسترسی‌های حافظه‌اش مداخله‌ای نمی‌کند (به خاطر جریمه‌ی کارایی که عاید می‌شود). سخت‌افزار، این کار را به شیوه‌های متفاوتی پیاده‌سازی می‌کند، چنان که در سراسر فصل نشان می‌دهیم. در اینجا، یک پیاده‌سازی ممکن را مطرح می‌کنیم.

در ابتدا لازم است که مطمئن شویم هر یک از پردازه‌ها فضای حافظه‌ی مجزایی دارند. فضای حافظه‌ی مجزا برای هر پردازه، پردازه‌ها را از یکدیگر محافظت کرده و مبنای داشتن پردازه‌های متعدد بار شده به داخل حافظه برای اجرای همروند است. برای تفکیک فضاهای حافظه، بایستی قادر باشیم محدوده‌ی آدرس‌های قانونی را که پردازه می‌تواند دسترسی کند تعیین کرده و اطمینان بدهیم که پردازه می‌تواند تنها همین آدرس‌های قانونی را دستیابی کند. این حفاظت را می‌توانیم با استفاده از دو ثبات، معمولاً یکی مبنا و دیگری حد، طوری که در شکل ۹-۱ نشان داده می‌شوند، فراهم کنیم. **ثبات مبنا**[1] کوچکترین آدرس فیزیکی قانونی حافظه را نگه می‌دارد؛ **ثبات حد**[2] اندازه‌ی بازه را مشخص می‌کند. به عنوان مثال، اگر ثبات مبنا برابر ۳۰۰۰۴۰ و ثبات حد برابر ۱۲۰۹۰۰ باشند، در این صورت برنامه می‌تواند به صورت قانونی، تمامی آدرس‌ها را از ۳۰۰۰۴۰ تا ۴۲۰۹۳۹ (بازه‌ی بسته) دستیابی کند.



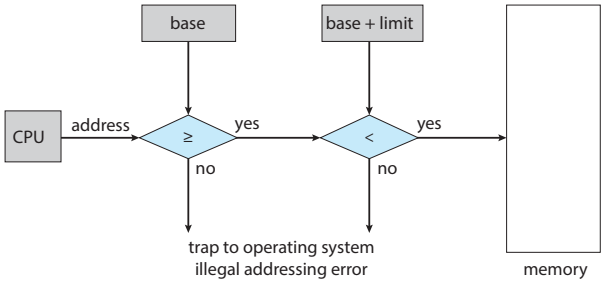
شکل ۹-۱ ثبات مبنا و ثبات حد، یک فضای آدرس منطقی تعریف می‌کنند.

سخت‌افزار پردازنده فضای حافظه را با مقایسه‌ی هر یک از آدرس‌های تولید شده در مد کاربر با مقادیر ثبات‌ها محافظت می‌کند. هرگونه اقدامی که توسط برنامه‌ی در حال اجرا در مد

1. Base register
2. Limit register

کاربر جهت دسترسی حافظه‌ی سیستم‌عامل یا حافظه‌ی سایر کاربران صورت بگیرد، تله‌ای به سیستم‌عامل را به دنبال دارد که این اقدام به عنوان یک خطای خرابکارانه تلقی می‌شود (شکل ۹-۲). این طرح از تغییر (تصادفی یا عمدی) کد یا ساختمان‌داده‌های سیستم‌عامل و یا سایر کاربران توسط برنامه‌ی کاربر جلوگیری می‌کند.

ثبات‌های مبنا و حد فقط توسط سیستم‌عامل بارگذاری می‌شوند که برای این کار از دستورالعمل ممتاز خاصی استفاده می‌کند. چون دستورالعمل‌های ممتاز فقط در مد هسته قابل اجرا هستند و از این نظر که فقط سیستم‌عامل در مد هسته اجرا می‌شود، تنها سیستم‌عامل می‌تواند ثبات‌های مبنا و حد را بارگذاری کند. این طرح، امکان تغییر مقادیر ثبات‌ها را برای سیستم‌عامل فراهم کرده و از تغییر محتوای ثبات‌ها توسط برنامه‌های کاربر جلوگیری می‌نماید.



شکل ۹-۲ حفاظت سخت‌افزاری آدرس با ثبات‌های مبنا و حد.

سیستم‌عامل با اجرا در مد هسته، دسترسی نامحدودی به حافظه‌ی سیستم‌عامل و کاربران خواهد داشت. این اختیارات به سیستم‌عامل اجازه می‌دهد تا برنامه‌های کاربران را در حافظه‌ی کاربران بارگذاری کند، در صورت بروز هرگونه خطایی، این برنامه‌ها را از حافظه بیرون بریزد، پارامترهای فراخوانی‌های سیستمی را دسترسی و تغییر بدهد، انجام I/O به و از حافظه‌ی کاربر و تدارک سرویس‌های فراوان دیگر. برای مثال، سیستم‌عامل یک سیستم چندپردازنده را در نظر بگیرید که بایستی برگردان متن‌ها را با ذخیره‌ی حالت یک پردازه از ثبات‌ها به داخل حافظه قبل از بارگذاری متن پردازه‌ی دیگر از حافظه‌ی اصلی به داخل ثبات‌ها، انجام بدهد.

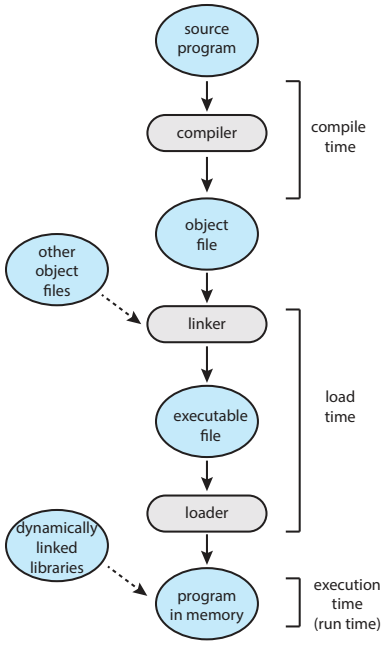
۹-۱-۲ پیوند آدرس

برنامه معمولاً به صورت فایل اجرایی دودویی بر روی دیسک مقیم است. برنامه به‌منظور اجرا بایستی داخل حافظه آورده شده و در قالب یک پردازه قرار بگیرد (چنان‌چه در بخش ۲-۵ بیان شد)، در اینجاست که برای اجرا بر روی یک پردازنده‌ی در دسترس آمادگی دارد. همچنان که پردازه اجرا می‌شود، دستورالعمل‌ها و داده‌ها را از حافظه دستیابی می‌کند. سرانجام، پردازه خاتمه یافته و فضای حافظه‌ی آن برای استفاده‌ی سایر پردازه‌ها پس‌گرفته می‌شود.

اکثر سیستم‌ها به پردازه‌ی کاربر اجازه می‌دهند در هر قسمتی از حافظه‌ی فیزیکی قرار بگیرد. بنابراین، هرچند که فضای آدرس کامپیوتر ممکن است از 00000 شروع شود، اولین آدرس

پردازه‌ی کاربر لزوماً 00000 نخواهد بود. بعداً خواهید دید که چگونه سیستم‌عامل به صورت واقعی، یک پردازه را در حافظه‌ی فیزیکی قرار می‌دهد.

در بیشتر موارد، برنامه‌ی کاربر پیش از آنکه اجرا شود مراحل متعددی را—برخی از آنها ممکن است اختیاری باشند—پشت سر می‌گذارد (شکل ۹-۳). آدرس‌ها در طی این مراحل ممکن است به شیوه‌های متفاوتی نمایش یابند. آدرس‌های برنامه‌ی مبدأ عموماً نمادین هستند (همچون متغیر count). کامپایلر نوعاً این آدرس‌های نمادین را به آدرس‌های جابجاپذیر (همچون «۱۴ بایت از شروع این ماژول») **پیوند**[1] می‌دهد. پیونددهنده یا بارگذار (بخش ۲-۵ را ببینید) هم به نوبت خود آدرس‌های جابجاپذیر را به آدرس‌های مطلق (نظیر ۷۴۰۱۴) پیوند خواهد زد. هر پیوند، نگاشتی از یک فضای آدرس به فضای آدرس دیگر است.



شکل ۹-۳ پردازش چندمرحله‌ای یک برنامه‌ی کاربر.

پیوند معمول دستورالعمل‌ها و داده‌ها به آدرس‌های حافظه در هر یک از مراحل زیر قابل انجام است:

- **زمان کامپایل**: اگر در زمان کامپایل بدانید که پردازه در کجای حافظه قرار خواهد گرفت، در این صورت **کد مطلق** قابل تولید است. برای مثال، اگر می‌دانید که پردازه‌ای از کاربر با شروع از مکان R مقیم می‌شود، در این صورت کد تولیدی کامپایلر از همان مکان شروع شده و گسترش خواهد یافت. اگر بعدها، مکان شروع

1. Bind

تغییر یابد، در این صورت لازم است که این کد دوباره کامپایل شود. برنامه‌های با قالب COM. در MS-DOS در زمان کامپایل پیوند داده می‌شوند.

- **زمان بارگذاری**: اگر در زمان کامپایل معلوم نشود که پردازه در کجای حافظه مقیم خواهد شد، در این صورت کامپایلر بایستی **کد جابجاپذیر**[1] تولید نماید. در این حالت، پیوند نهایی تا زمان بارگذاری به تأخیر می‌افتد. اگر آدرس شروع تغییر یابد، فقط لازم داریم که کد کاربر را دوباره بارگذاری کنیم تا این تغییرات لحاظ شوند.
- **زمان اجرا**: اگر قرار است که پردازه در طول اجرای خود بتواند از یک قطعه‌ی حافظه به قطعه‌ای دیگر جابجا شود، در این صورت بایستی پیوند را تا زمان اجرا به تأخیر بیندازیم. همان‌گونه که در بخش ۹-۱-۳ بحث خواهد شد، برای اعمال این طرح بایستی سخت‌افزار خاصی فراهم شود. اکثر سیستم‌عامل‌ها از این روش استفاده می‌کنند.

بخش اعظم این فصل به چگونگی پیاده‌سازی کارآمد این نوع پیوندها در یک سیستم کامپیوتری اختصاص داشته و پشتیبانی سخت‌افزاری مناسب آنها نیز مورد بحث قرار می‌گیرد.

۹-۱-۳ فضای آدرس منطقی در برابر فضای آدرس فیزیکی

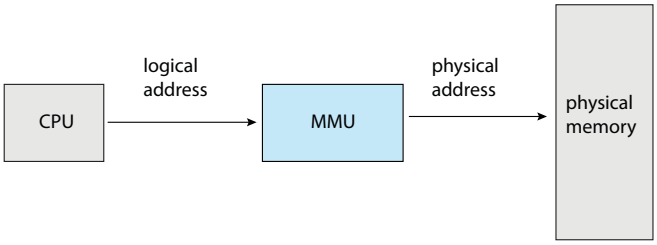
آدرسی که توسط پردازنده تولید می‌شود، معمولاً به نام **آدرس منطقی** رجوع می‌شود، در حالی که آدرسی که واحد حافظه می‌بیند—یعنی، آدرسی که داخل **ثبات آدرس حافظه** بارگذاری می‌شود—بیشتر به **آدرس فیزیکی** شهرت دارد.

پیوند آدرس‌های در زمان کامپایل و یا بارگذاری، آدرس‌های منطقی و فیزیکی یکسانی تولید می‌نمایند. با این وجود، در طرح پیوند آدرس زمان اجرا، آدرس‌های منطقی و فیزیکی با هم فرق دارند. در اینجا، به آدرس منطقی تحت عنوان **آدرس مجازی**[2] رجوع می‌کنیم. در این کتاب از *آدرس منطقی* و *آدرس مجازی* به جای هم استفاده می‌کنیم. مجموعه‌ی تمامی آدرس‌های منطقی تولید شده توسط برنامه، **فضای آدرس منطقی** است. مجموعه‌ی تمامی آدرس‌های فیزیکی متناظر این آدرس‌های منطقی نیز **فضای آدرس فیزیکی** به حساب می‌آید. بنابراین، فضای آدرس‌های منطقی و فیزیکی در طرح پیوند آدرس زمان اجرا با هم فرق دارند.

نگاشت زمان اجرای آدرس‌های مجازی به فیزیکی توسط یک دستگاه سخت‌افزاری به نام **واحد مدیریت حافظه**[3] (**MMU**) (شکل ۹-۴) انجام می‌شود. همان‌گونه که در بخش ۹-۲ تا بخش ۹-۳ بحث خواهیم کرد، برای انجام چنین نگاشتی می‌توانیم از روش‌های متفاوت بسیاری که در این زمینه وجود دارند یکی را انتخاب کنیم. فعلاً برای نمایش این نگاشت از یک طرح ساده‌ی MMU که حالت کلی‌تری از طرح ثبات مبنای مطرح شده در بخش ۹-۱-۱ می‌باشد، استفاده می‌کنیم. در اینجا، به ثبات مبنا، **ثبات تغییر مکان**[4] گفته می‌شود. در زمان ارسال هر یک از آدرس‌های تولیدی پردازه‌ی کاربر به حافظه، مقدار ثبات تغییر مکان افزوده می‌شود (شکل ۹-۵ را ببینید). به عنوان مثال، اگر مبنا در ۱۴۰۰۰ باشد، در این صورت اگر کاربر بخواهد آدرس مکان ۰ را دسترسی کند

1. Relocatable code
2. Virtual address
3. Memory Management Unit (MMU)
4. Relocation register

به صورت پویا به مکان ۱۴۰۰۰ تغییر مکان خواهد یافت؛ دسترسی مکان ۳۴۶ به مکان ۱۴۳۴۶ نگاشته می‌شود.



شکل ۹-۴ واحد مدیریت حافظه (MMU).

برنامه‌ی کاربر هرگز آدرس‌های فیزیکی واقعی را دسترسی نمی‌کند. برنامه می‌تواند اشاره‌گری به مکان ۳۴۶ ایجاد کرده، آن را در حافظه ذخیره، دستکاری و با سایر آدرس‌ها مقایسه کند (در تمامی این موارد، عدد ۳۴۶ را می‌بیند). تنها زمانی که به عنوان آدرس حافظه استفاده می‌شود (احتمالاً با بارگذاری یا ذخیره‌ی غیر مستقیم) نسبت به ثبات مبنا تغییر مکان می‌یابد. برنامه‌ی کاربر با آدرس‌های منطقی سر و کار دارد. سخت‌افزار نگاشت حافظه، آدرس‌های منطقی را به آدرس‌های فیزیکی تبدیل می‌کند. این شکل از پیوند زمان اجرا در بخش ۹-۱-۲ مورد بحث قرار می‌گیرد. مکان نهایی آدرسی از حافظه که رجوع می‌شود مادامی که مراجعه نداشته، مشخص نمی‌شود.

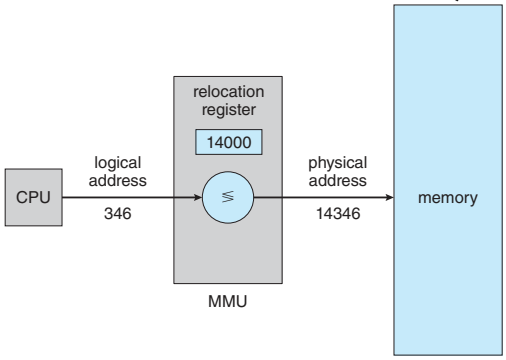
اکنون دو نوع آدرس متفاوت داریم که عبارتند از: آدرس‌های منطقی (در بازه‌ی 0 تا *max*) و آدرس‌های فیزیکی (در بازه‌ی $R + 0$ تا $R + max$ با مقدار مبنای R). برنامه‌ی کاربر فقط آدرس‌های منطقی را تولید کرده و تصور می‌کند که پردازه در مکان‌های 0 تا *max* حافظه اجرا می‌شود. با این وجود، این آدرس‌های منطقی بایستی پیش از آنکه استفاده شوند به آدرس‌های فیزیکی نگاشته شوند. مفهوم فضای آدرس منطقی که به فضای آدرس فیزیکی مجزایی پیوند می‌خورد محور مدیریت درست حافظه می‌باشد.

۹-۱-۴ بارگذاری پویا

در مباحثی که تاکنون داشتیم، پیش از آنکه پردازه اجرا شود لازم بود کل برنامه و داده‌های آن تماماً در حافظه‌ی فیزیکی قرار بگیرند. بنابراین، اندازه‌ی یک پردازه به اندازه‌ی حافظه‌ی فیزیکی محدود می‌شود. برای این که بهره‌وری فضای حافظه بهتر شود، می‌توانیم از **بارگذاری پویا** استفاده کنیم. در بارگذاری پویا، مادامی که یک روال فراخوانی نشده، بارگذاری نمی‌شود. تمامی روال‌ها بر روی دیسک با قالب بارگذاری جابجاپذیر نگه‌داری می‌شوند. برنامه‌ی اصلی در داخل حافظه بارگذاری شده و اجرا می‌شود. زمانی که روالی نیاز به فراخوانی روال دیگری دارد، روال فراخواننده ابتدا بررسی می‌کند تا ببیند آیا روالی که فراخوانی می‌کند بارگذاری شده است. اگر چنین نبود، بارگذار پیوندی جابجاپذیر فراخوانی می‌شود تا روال مطلوب را داخل حافظه بار کرده و جداول آدرس برنامه را برای لحاظ این تغییر به روز نماید. سپس، کنترل به

روالی که به تازگی بار شده است منتقل می‌شود.

مزیت بارگذاری پویا این است که یک روال فقط زمانی بارگذاری می‌شود که مورد نیاز است. این روش، مخصوصاً زمانی کارآمد است که انبوهی از کد داریم که برای مواجهه با وقایع نادر لازمند، نظیر روال‌های رسیدگی به خطا. در چنین شرایطی، هرچند که اندازه‌ی کل برنامه می‌تواند بزرگ باشد، اما بخشی که استفاده می‌شود (و بارگذاری هم می‌شود)، می‌تواند بسیار کوچک‌تر باشد.



شکل ۹–۵ تغییر مکان پویا با استفاده از یک ثبات تغییر مکان.

بارگذاری پویا نیاز خاصی به پشتیبانی سیستم‌عامل ندارد. این مسؤولیت کاربران است که برنامه‌های خود را طوری طراحی نمایند تا از چنین روشی بهره ببرد. با این وجود، سیستم‌عامل‌ها می‌توانند با فراهم‌کردن روال‌های کتابخانه‌ای، برنامه‌نویس را در جهت پیاده‌سازی بارگذاری پویا کمک کنند.

۹–۱–۵ اتصال پویا و کتابخانه‌های مشترک

کتابخانه‌های اتصالی پویا[1] کتابخانه‌های سیستمی هستند که در زمان اجرای برنامه‌های کاربر به آنها متصل می‌شوند (به شکل ۹–۳ رجوع کنید). برخی سیستم‌عامل‌ها فقط **اتصال ایستا**[2] را پشتیبانی می‌کنند که در آن با کتابخانه‌های سیستمی همانند سایر ماژول‌های شئ رفتار شده و توسط بارگذار با تصویر دودویی برنامه ترکیب می‌شوند. اتصال پویا در مقایسه، مشابه بارگذاری پویا می‌باشد. هرچند که در اینجا، به جای بارگذاری، این اتصال است که تا زمان اجرا به تعویق می‌افتد. معمولاً، این امکان با کتابخانه‌های سیستمی، همچون کتابخانه‌ی استاندارد زبان C مورد استفاده قرار می‌گیرد. اگر این امکان نباشد، هر یک از برنامه‌های روی سیستم بایستی یک کپی از کتابخانه‌ی زبان خود (یا حداقل روال‌هایی که برنامه رجوع می‌کند) را نیز در تصویر اجرایی خود لحاظ کنند. این نیازمندی نه تنها اندازه‌ی یک تصویر اجرایی را افزایش می‌دهد بلکه می‌تواند حافظه‌ی اصلی را نیز هدر بدهد. مزیت دوم DLLها این است که این

1. Dynamically linked libraries
2. Static linking

کتابخانه‌ها می‌توانند مابین پردازه‌های متعددی به‌اشتراک گذاشته شوند، طوری که تنها یک نمونه از DLL در حافظه‌ی اصلی باشد. به‌همین دلیل، DLLها به **کتابخانه‌های مشترک** نیز معروف هستند و به‌صورت گسترده در سیستم‌های ویندوز و لینوکس استفاده می‌شوند.

موقعی که یک برنامه به یک روالی رجوع می‌کند که در یک کتابخانه‌ی پویاست، بارگذار DLL را پیدا کرده، در صورت لزوم به داخل حافظه بارگذاری می‌کند. در ادامه، آدرس‌ها را تطبیق می‌دهد تا توابع رجوع‌کننده به کتابخانه‌ی پویا به مکانی از حافظه که DLL ذخیره می‌شود ارجاع کنند.

کتابخانه‌های اتصالی پویا می‌توانند با به‌روزرسانی‌های کتابخانه (همچون ترمیم عیب‌ها[1]) گسترش پیدا کنند. به‌علاوه، کتابخانه می‌تواند با نسخه‌ی جدیدی جایگزین شود و تمامی برنامه‌هایی که به کتابخانه رجوع دارند به طور خودکار از این نسخه‌ی جدید استفاده نمایند. بدون داشتن اتصال پویا، تمامی این قبیل برنامه‌ها برای دستیابی کتابخانه‌ی جدید لازم دارند که اتصال خود را تجدید نمایند. لذا، برنامه‌ها به صورت تصادفی، نسخه‌های جدید ناسازگار کتابخانه‌ها را اجرا نخواهند کرد، چرا که اطلاعات مربوط به نسخه در هر دوی برنامه و کتابخانه نیز لحاظ می‌شود. حتی این امکان وجود دارد که نسخه‌های متعددی از کتابخانه‌ها در حافظه بارگذاری شوند و از این رو، هر یک از برنامه‌ها با استفاده از اطلاعات نسخه‌ی خود می‌توانند تصمیم بگیرند که از کدام نسخه استفاده نمایند. نسخه‌هایی که تغییرات اندکی دارند شماره‌ی نسخه‌ی خود را حفظ می‌کنند، حال آنکه با تغییرات اساسی، شماره‌ی نسخه افزایش می‌یابد. بنابراین، فقط برنامه‌هایی که با نسخه‌ی جدید کتابخانه کامپایل می‌شوند از تغییرات ناسازگار موجود آن متأثر خواهند شد. سایر برنامه‌هایی که اتصال آنها قبل از نصب کتابخانه‌ی جدید صورت گرفته است به استفاده از کتابخانه‌ی قدیمی‌تر ادامه می‌دهند.

برعکس بارگذاری پویا، اتصال پویا و کتابخانه‌های مشترک عموماً به کمک سیستم‌عامل نیاز دارند. اگر قرار باشد که پردازه‌های داخل حافظه از یکدیگر محافظت شوند، در این صورت سیستم‌عامل، تنها نهادی است که می‌تواند بررسی کند که ببیند آیا روال مورد نیاز در فضای حافظه‌ی پردازه‌ی دیگری قرار دارد و یا به پردازه‌های متعددی اجازه بدهد که آدرس‌های یکسانی از حافظه را دستیابی کنند. این مفهوم، همین‌طور چگونگی به اشتراک‌گذاری DLLها توسط پردازه‌های متعدد را با تمرکز بیشتری در بخش ۹-۳-۴، موقع بحث صفحه‌بندی بررسی می‌کنیم.

۹-۲ تخصیص همجوار حافظه

حافظه‌ی اصلی بایستی هر دوی سیستم‌عامل و پردازه‌های مختلف کاربر را تحت پوشش قرار بدهد. بنابراین، لازم داریم که بخش‌های حافظه‌ی اصلی را تا حد ممکن به شیوه‌ای کارآمد و مؤثر تخصیص بدهیم. این بخش، یک روش قدیمی، یعنی تخصیص همجوار حافظه[2] را تشریح می‌نماید.

1. Bug fixes
2. Contiguous memory allocation